Johns Hopkins Engineering

Software Defined Radio

Live Lab 10

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

Goals:

- This Live Lab will:
  - Develop skills with Error Detection and Correction

Outline

1. Questions from Last Week’s Homework
2. Background
3. Hamming(7,4) Examples
4. Hamming(7,4) Simulation, Correction
5. Hamming(7,4) Simulation, Detection
6. CCITT CRC-16 Simulation

Last Week’s Homework

Last Week’s Homework

- Q & A

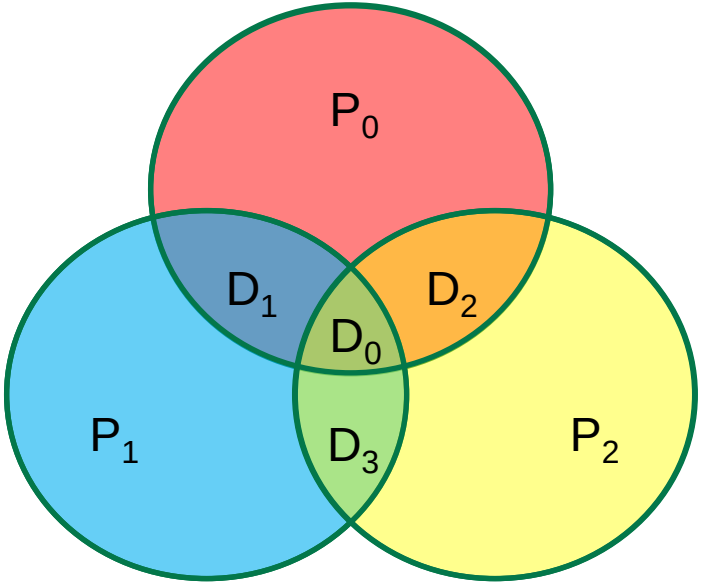
Hamming(7,4)

Hamming(7,4) (1)

- Linear Block Code
- The code rate is 4/7
- Provides a parity check
- Message bits are concatenated with parity bits to form code words.

Hamming(7,4) (2)

- The Hamming(7,4) Venn Diagram



Hamming(7,4) (3)

- The Hamming(7,4) code
  - can detect up to two errors
  - can correct up to one error

```
[h_parity_matrix,g_generator_matrix] = hammgen(3);
the_syndrome_table = syndtable(h_parity_matrix);


data_bits = [1 1 1 1];
hamming_coded_bits = mod(data_bits*g_generator_matrix,2)
syndrome_result = mod(hamming_coded_bits*h_parity_matrix.',2);
```

Hamming(7,4) Test (1)

- A Syndrome of Zero

```
coded_bit_error_vector = [0 0 0 0 0 0 0];
hamming_coded_bits_in_error = ...
mod(hamming_coded_bits+coded_bit_error_vector,2);
syndrome_result = mod(hamming_coded_bits_in_error*h_parity_matrix.',2);
syndrome_decimal = bi2de(syndrome_result,'left-msb');
correction_vector = the_syndrome_table(1+syndrome_decimal,:);
corrected_hamming_coded_bits_no_error = ...
mod(correction_vector+hamming_coded_bits_in_error,2)
```

No Errors Detected, Syndrome is Zero

Hamming(7,4) Test (2)

- A Non-Zero Syndrome, 1 Error

```
coded_bit_error_vector = [0 1 0 0 0 0 0];
hamming_coded_bits_in_error = ...
mod(hamming_coded_bits+coded_bit_error_vector,2);
syndrome_result = mod(hamming_coded_bits_in_error*h_parity_matrix.',2);
syndrome_decimal = bi2de(syndrome_result,'left-msb');
correction_vector = the_syndrome_table(1+syndrome_decimal,:);
corrected_hamming_coded_bits_one_error =...
mod(correction_vector+hamming_coded_bits_in_error,2)
```

One Error, Errors Detected, Syndrome is Non-Zero

Hamming(7,4) Test (3)

- A Non-Zero Syndrome, 2 Errors

```
coded_bit_error_vector = [0 1 0 1 0 0 0];
hamming_coded_bits_in_error = ...
mod(hamming_coded_bits+coded_bit_error_vector,2);
syndrome_result = mod(hamming_coded_bits_in_error*h_parity_matrix.',2);
syndrome_decimal = bi2de(syndrome_result,'left-msb');
correction_vector = the_syndrome_table(1+syndrome_decimal,:);
corrected_hamming_coded_bits_two_errors = ...
rem(correction_vector+hamming_coded_bits_in_error,2)
```

Two Error, Errors Detected, Syndrome is Non-Zero, but cannot correct

Hamming(7,4) Test (4)

- A Non-Zero Syndrome, 3 Errors

```
coded_bit_error_vector = [0 1 0 1 0 1 0];
hamming_coded_bits_in_error = ...
mod(hamming_coded_bits+coded_bit_error_vector,2);
syndrome_result = mod(hamming_coded_bits_in_error*h_parity_matrix.',2);
syndrome_decimal = bi2de(syndrome_result,'left-msb');
correction_vector = the_syndrome_table(1+syndrome_decimal,:);
corrected_hamming_coded_bits_three_errors_v1 = ...
rem(correction_vector+hamming_coded_bits_in_error,2)
```

Three Error, Errors Detected, Syndrome is Non-Zero, cannot correct

Hamming(7,4) Test (5)

- A Syndrome of Zero, 3 Errors

```
coded_bit_error_vector = [1 1 0 1 0 0 0];
hamming_coded_bits_in_error = ...
mod(hamming_coded_bits+coded_bit_error_vector,2);
syndrome_result = mod(hamming_coded_bits_in_error*h_parity_matrix.',2);
syndrome_decimal = bi2de(syndrome_result,'left-msb');
correction_vector = the_syndrome_table(1+syndrome_decimal,:);
corrected_hamming_coded_bits_three_errors_v2 = ...
rem(correction_vector+hamming_coded_bits_in_error,2)
```

Three Error, No Errors Detected, Syndrome is Zero

Hamming(7,4) Summary

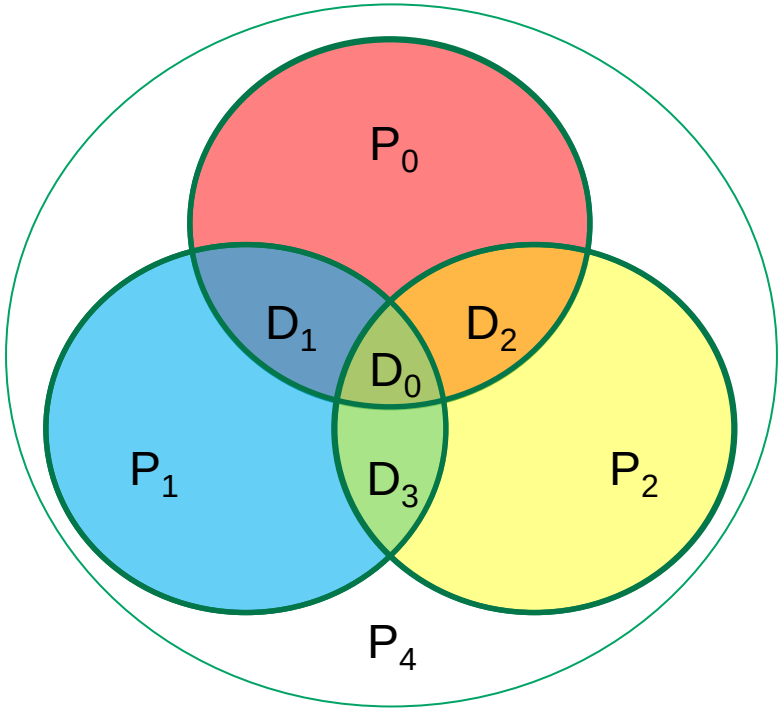
- Guaranteed to Detect any two-bit error
  - *But cannot distinguish between 1 and 2 errors*

- Guaranteed to Correct any one-bit error

SECDED

- The SECDED, Hamming(8,4),Venn Diagram

- Can distinguish between 1 and 2 errors
- Can only correct 1 error.



Hamming(7,4) Simulation, Correction

Simulation Objectives

- Simulate BER with Hamming(7,4) Correction
- Compare to Uncorrected BER

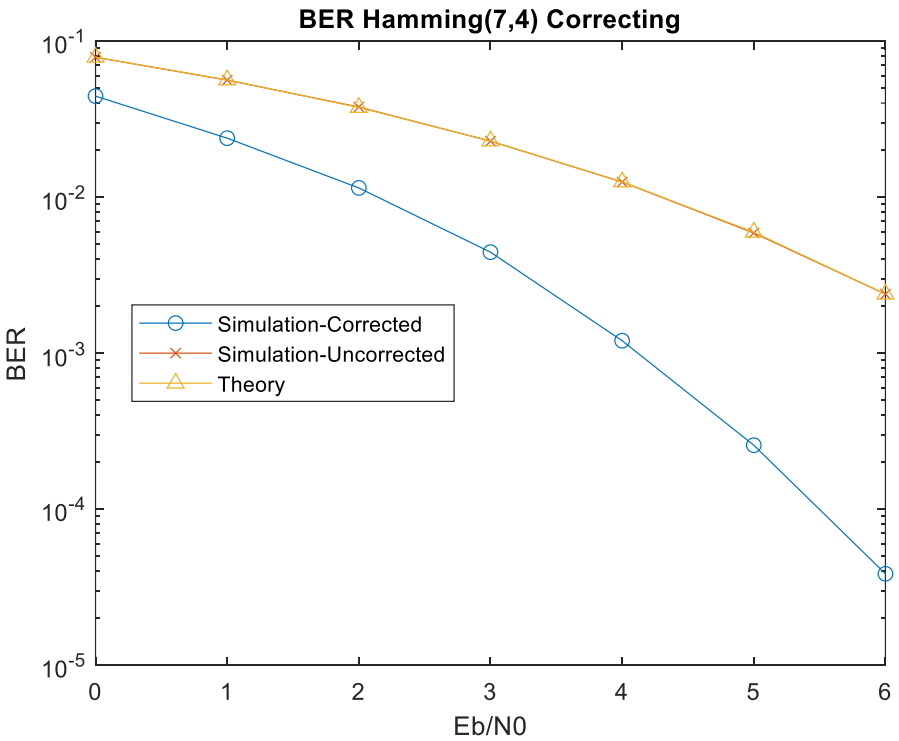
The Simulation

```
[h_parity_matrix,g_generator_matrix] = hammgen(3);
the_syndrome_table = syndtable(h_parity_matrix);

num_frames = 100000;
num_data_bits = num_frames*4;
num_coded_bits = num_frames*7;

awgn_vector = 0:6;  %in dB

ber_before_correction = zeros(1,length(awgn_vector));
ber_after_correction = zeros(1,length(awgn_vector));
probability_bit_error=zeros(1,length(awgn_vector));
```

```
for idx=1:length(awgn_vector)
    after_fec = zeros(num_frames,1);
    before_fec = zeros(num_frames,1);
    for k = 1:num_frames
        data_bits = randi([0 1],1,4); % Generate binary data
        encoded_bits = ...
            mod(data_bits*g_generator_matrix,2); % Append Parity bits
        modulated_data = pskmod(encoded_bits,2); % BPSK modulate
        noise_vector = randn(1,7)+1.0i*randn(1,7);
        noise_vector = noise_vector*sqrt(1/2);
        noise_vector = ...
            noise_vector*sqrt(10^(-awgn_vector(idx)/10));
        rxSig = modulated_data + noise_vector;  % AWGN channel, SNR dB
        demodulated_data = pskdemod(rxSig,2);  % BPSK demodulate
        syndrome = mod(h_parity_matrix*demodulated_data.',2);
        syndrome_de = bi2de(syndrome.','left-msb');
        correction_vector = the_syndrome_table(1+syndrome_de,:);
        if ~isequal(demodulated_data,encoded_bits)
            before_fec(k)=sum(abs(demodulated_data-encoded_bits));
        end
        demodulated_data = rem(correction_vector+demodulated_data,2);
        if ~isequal(demodulated_data,encoded_bits)
            after_fec(k)=sum(abs(demodulated_data-encoded_bits));
        end
    end
    ber_before_correction(idx) = sum(before_fec)/num_coded_bits;
    ber_after_correction(idx) = sum(after_fec)/num_coded_bits;
    probability_bit_error(idx)=qfunc(sqrt(2*10^(awgn_vector(idx)/10)));
end
```

```
figure
semilogy(awgn_vector,ber_after_correction,'-o')
hold on
semilogy(awgn_vector,ber_before_correction,'-x')
semilogy(awgn_vector,probability_bit_error,'-^')
hold off
title('BER Hamming(7,4) Correcting')
xlabel('Eb/N0')
ylabel('BER')
legend('Simulation-Corrected','Simulation-Uncorrected',...
    'Theory','location','best')
```

Results


BER Hamming(7,4) Correcting
BER
Eb/N0
10^{-1}
10^{-2}
10^{-3}
10^{-4}
10^{-5}
0
1
2
3
4
5
6
Simulation-Corrected
Simulation-Uncorrected
Theory

Analysis

- Hamming(7,4) error correction provided a significant improvement in BER performance.

- There are two issues:
  - Energy per bit was kept constant. The coded packet transmits for 7/4 times the un-coded packet. That means the coded transmission uses 7/4 times as much energy overall.
  - There were still bit errors. Any bits errors in the packet means the packet must be discarded. We need a means to detect bit errors.

Hamming(7,4) Simulation, Detection

Detecting Errors, but not correcting them

- In this exercise we will use the Hamming codes to detect the presence of bit errors.
- If there is an uncorrected bit error, then the entire packet containing that bit error is in error.
- Therefore we will measure the Packet Error Rate.

The Simulation

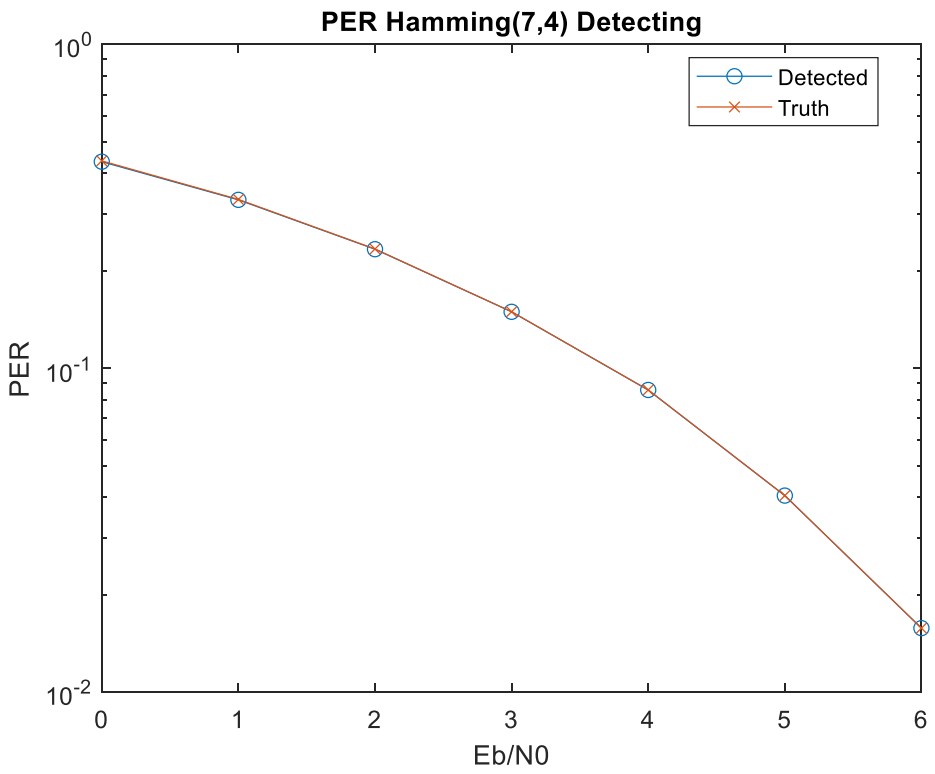
```
[h_parity_matrix,g_generator_matrix] = hammgen(3);
the_syndrome_table = syndtable(h_parity_matrix);


num_frames = 100000;
num_data_bits = num_frames*4;
num_coded_bits = num_frames*7;

awgn_vector = 0:6; %in dB

numFrames = 100000;
awgnvec=0:6;
PER_detected=zeros(1,length(awgnvec));
PER_truth=zeros(1,length(awgnvec));
```

```
for idx=1:length(awgnvec)
    detected_error = zeros(numFrames,1);
    truth_error = zeros(numFrames,1);
    for k = 1:numFrames
        data_bits = randi([0 1],4,1); % Generate binary data
        encoded_data = ...
            mod(data_bits.'*g_generator_matrix,2); % Append Parity bits
        modulated_data = pskmod(encoded_data,2); % BPSK modulate
        noise_vector = randn(1,7)+1.0i*randn(1,7);
        noise_vector = noise_vector*sqrt(1/2);
        noise_vector = ...
            noise_vector*sqrt(10^(-awgn_vector(idx)/10));
        rxSig = modulated_data + noise_vector;  % AWGN channel, SNR dB
        demodulated_data = pskdemod(rxSig,2); % BPSK demodulate
        syndrome = mod(h_parity_matrix*demodulated_data.',2);
        if sum(syndrome)>0
            detected_error(k)=1;
        end
        if ~isequal(demodulated_data,encoded_data)
            truth_error(k)=1;
        end
    end
    PER_detected(idx) = sum(detected_error)/numFrames;
    PER_truth(idx) = sum(truth_error)/numFrames;
end
```

```
figure
semilogy(awgnvec,PER_detected,'-o')
hold on
semilogy(awgnvec,PER_truth,'-x')
hold off
title('PER Hamming(7,4) Detecting')
xlabel('Eb/N0')
ylabel('PER')
legend('Detected','Truth',...
'location','best')
figure
plot(awgnvec,PER_truth-PER_detected)
xlabel('SNR')
ylabel('Difference in PER (Actual-Measured)')
```

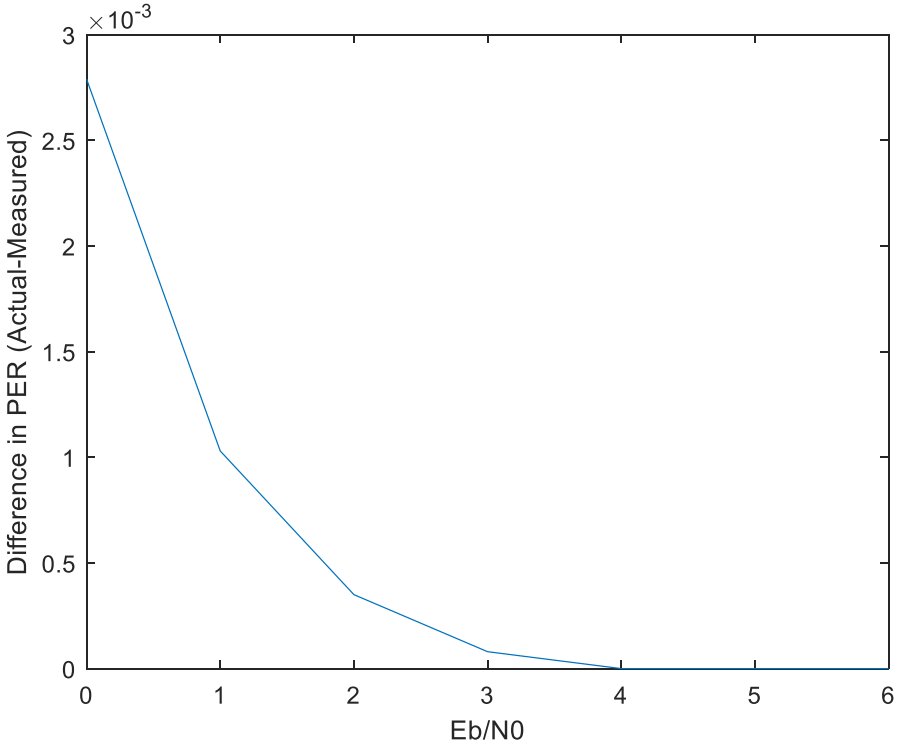
Results (1)

- How well can we detect a packet error?



Results (2)

- How well can we detect a packet error?



Analysis

- Hamming(7,4) can detect errors in packets
  - As SNR decreases, the likelihood of more than two errors in a packet increases.
  - This means Hamming(7,4) begins to fail to detect errors in packets.

CRC Simulation, Detection

The Simulation, 32 bits of Message

```
crcgenerator = comm.CRCGenerator(...
    'Polynomial','X^16 + X^12 + X^5 + 1',...
    'InitialConditions',1,...
    'DirectMethod',true,...
    'FinalXOR',1);
crcdetector = comm.CRCDetector(...
    'Polynomial','X^16 + X^12 + X^5 + 1',...
    'InitialConditions',1,...
    'DirectMethod',true,...
    'FinalXOR',1);

numFrames = 100000;
awgnvec = 0:6;
message_length = 32;
PER_detected=zeros(1,length(awgnvec));
PER_truth=zeros(1,length(awgnvec));
```

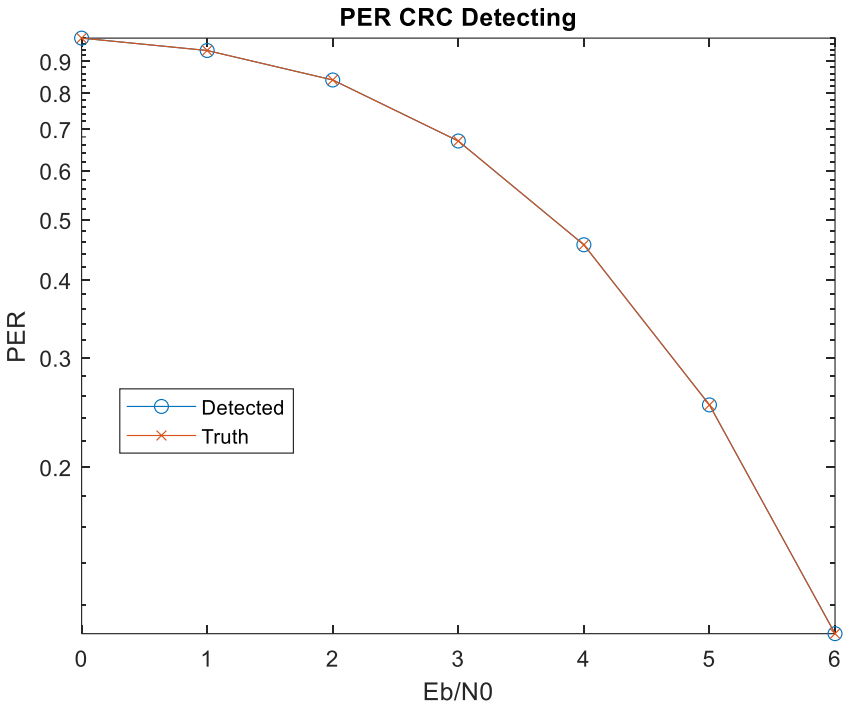
```
for idx=1:length(awgnvec)
    detected_error = zeros(numFrames,1);
    truth_error = zeros(numFrames,1);
    for k = 1:numFrames
        data = randi([0 1],message_length,1); % Generate binary data
        encoded_data = crcgenerator(data); % Append CRC bits
        modulated_data = pskmod(encoded_data,2); % BPSK modulate
        signal_length = length(modulated_data);
        noise_vector = randn(1,signal_length)+1.0i*randn(1,signal_length);
        noise_vector = noise_vector*sqrt(1/2);
        noise_vector = ...
            noise_vector*sqrt(10^(-awgn_vector(idx)/10));
        rxSig = modulated_data + noise_vector;  % AWGN channel, SNR dB
        demodulated_data = pskdemod(rxSig,2); % BPSK demodulate
        [~,detected_error(k)] = ...
            crcdetector(demodulated_data); % Detect CRC errors
        if ~isequal(demodulated_data,encoded_data)
            truth_error(k)=1;
        end
    end
    PER_detected(idx) = sum(detected_error)/numFrames;
    PER_truth(idx) = sum(truth_error)/numFrames;
end
```

```
figure
semilogy(awgnvec,PER_detected,'-o')
hold on
semilogy(awgnvec,PER_truth,'-x')
hold off
title('PER CRC Detecting')
xlabel('Eb/N0')
ylabel('PER')
legend('Detected','Truth',...
    'location','best')

figure
plot(awgnvec,PER_truth-PER_detected)
xlabel('SNR')
ylabel('Difference in PER (Actual-Measured)')
```

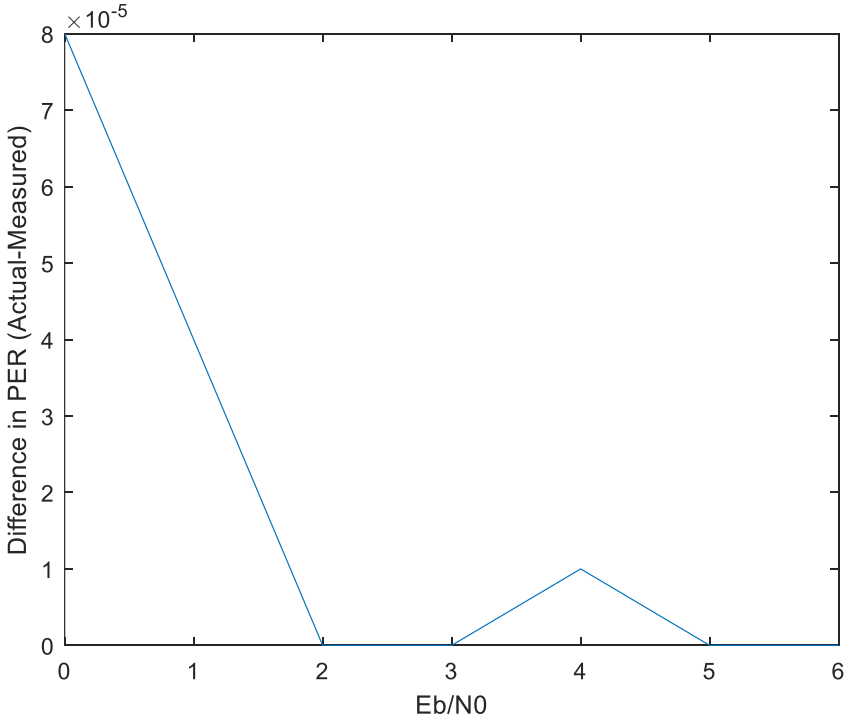
Results (1)

- Note that packet errors are more common than before.

- Why?



Results (2)

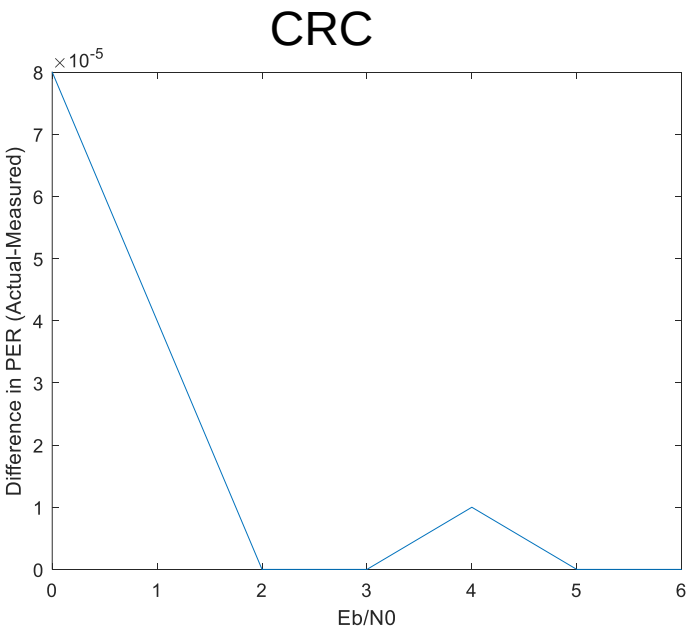


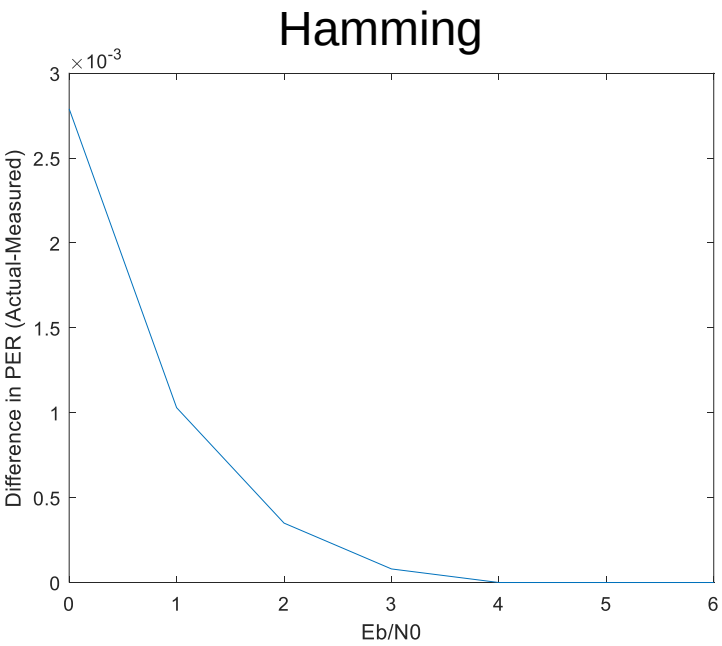
These errors are very small, and your results may vary

Increase simulation length to improve precision

A longer simulation will run too long for this exercise

Which Is More Accurate?





Analysis

- The CRC Detection has almost no missed detections.

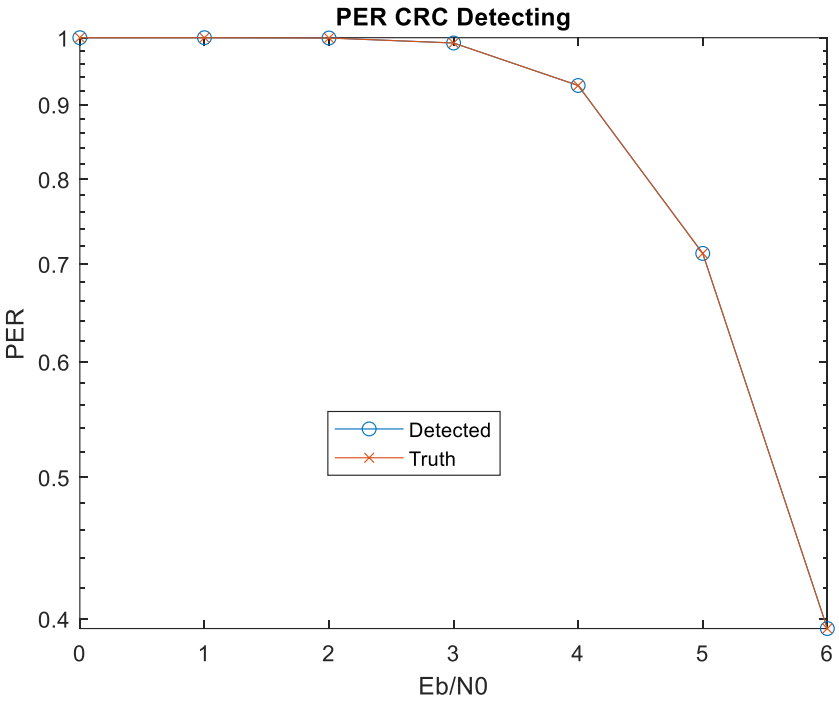
- This simulation exhibits a higher PER, both in detection and truth data.
  - This is because the packet lengths are longer,
  - The Hamming (7,4) code works on frames 7 bits long.
  - This new scheme has a packet length of 32 bits.

The Simulation, 192 bits of message

- Change the message length from 32 to 192
- Run the simulation

Results (1)

- Note that packet errors are more common than before.
- Why?



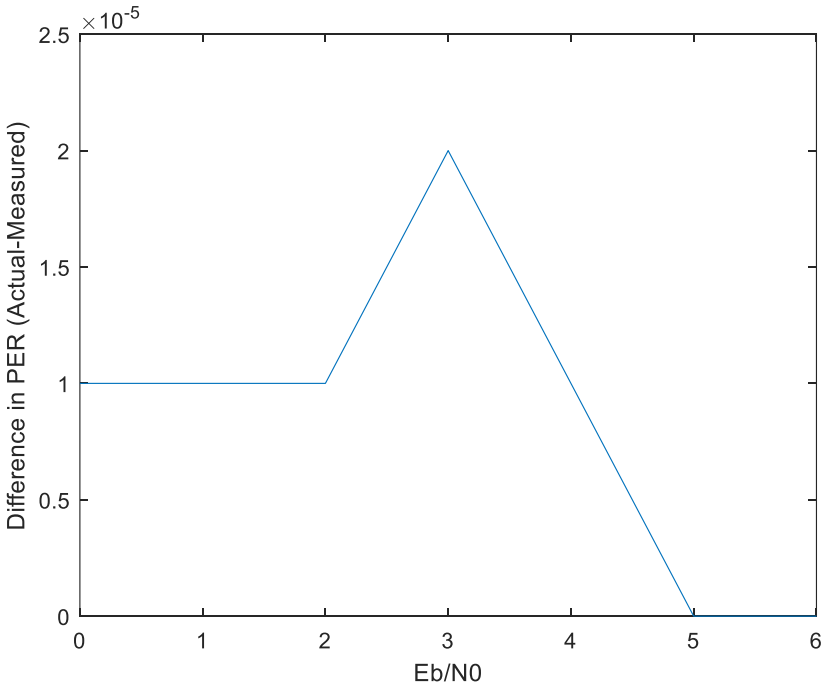
Results (2)

- Accuracy is still very high

  These errors are very small, and your results may vary

  Increase simulation length to improve precision

  A longer simulation will run too long for this exercise



Analysis

- The PER gets worse as packet length increases
  - Greater chance for any one bit to be in error
  - If any one bit is in error, the whole packet is dumped

JOHNS HOPKINS
WHITING SCHOOL
of ENGINEERING

© The Johns Hopkins University 2020, All Rights Reserved.